

OPTIMIZĂRI – TEMA 1 – PROBLEMĂ DE REGRESIE

PREDICȚIA CONCENTRAȚIEI DE MONOXID DE CARBON DIN AERUL RESPIRAT

Dragoș – Alexandru Duduc, grupa 324AA, funcția de activare nr.41

Informații despre baza de date aleasă:

Am ales setul de date “Air Quality” de pe UC Irvine Machine Learning Repository. Această bază de date pune la dispoziție în jur de 9000 de măsurători la momente diferite de timp, pe un câmp dintr-un oraș italian. Sunt măsurate concentrațiile de monoxid de carbon (CO), benzen (C_6H_6), dioxid de azot (NO_2), ozon (O_3) etc. Baza de date nu este completă, măsurătoarea lipsă fiind marcată prin valoarea -200.

1	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	T	RH	AH
2	10-03-04	18:00:00	2.6	1360	150	11.9	1046	166	1056	113	1692	1268	13.6	48.9	0.7578
3	10-03-04	19:00:00	2	1292	112	9.4	955	103	1174	92	1559	972	13.3	47.7	0.7255
4	10-03-04	20:00:00	2.2	1402	88	9.0	939	131	1140	114	1555	1074	11.9	54.0	0.7502
5	10-03-04	21:00:00	2.2	1376	80	9.2	948	172	1092	122	1584	1203	11.0	60.0	0.7867
6	10-03-04	22:00:00	1.6	1272	51	6.5	836	131	1205	116	1490	1110	11.2	59.6	0.7888
7	10-03-04	23:00:00	1.2	1197	38	4.7	750	89	1337	96	1393	949	11.2	59.2	0.7848
8	11-03-04	0:00:00	1.2	1185	31	3.6	690	62	1462	77	1333	733	11.3	56.8	0.7603
9	11-03-04	1:00:00	1	1136	31	3.3	672	62	1453	76	1333	730	10.7	60.0	0.7702
10	11-03-04	2:00:00	0.9	1094	24	2.3	609	45	1579	60	1276	620	10.7	59.7	0.7648

Am început prin a elimina primele două coloane, neavând nevoie de momentele de timp la care au fost realizate. Am ales în jur de 350 de exemple (linii din tabel) și am ales drept vector de etichete măsurătorile privind concentrația de CO. Cu alte cuvinte, programul va realiza predicția concentrației de CO din aerul respirat pe baza concentrațiilor celorlalți compuși chimici. La următorul pas am eliminat din baza de date toate exemplele pentru care lipsesc măsurătorile de CO și am completat măsurătorile lipsă la fiecare caracteristică cu valoarea medie a caracteristicii din baza de date de antrenare (nu și cea de testare, pentru a nu avea acces la măsurători “necunoscute”). În final, a fost nevoie să standardizez baza de date de antrenare la medie 0 și deviație standard 1. Am aplicat aceeași transformare și pe baza de date de testare, și am adăugat amândurora coloana suplimentară 1_N pentru bias.

Rezolvarea problemei de minimizare:

Realizăm o rețea neurală superficială, adică cu un singur strat intermediar care conține un număr de neuroni $m = 20$ fixat. Pentru găsirea celor mai bune ponderi ale rețelei neurale, rezolvăm problema de minimizare fără constrângeri:

$$\min_{x \in \mathbb{R}^m, X \in \mathbb{R}^{(n+1) \times m}} L(e, g(\bar{A}X) x)$$

unde ABar este baza de date de antrenare extinsă cu coloana 1_N, X este matricea ponderilor de la stratul de intrare la stratul ascuns al rețelei neurale, x este vectorul de ponderi de la stratul ascuns la ieșire, e este vectorul de etichete, g(z) este funcția 41. $g(z) = az^3$ (a = 1 fixat) și am ales loss function-ul de eroare medie pătrată:

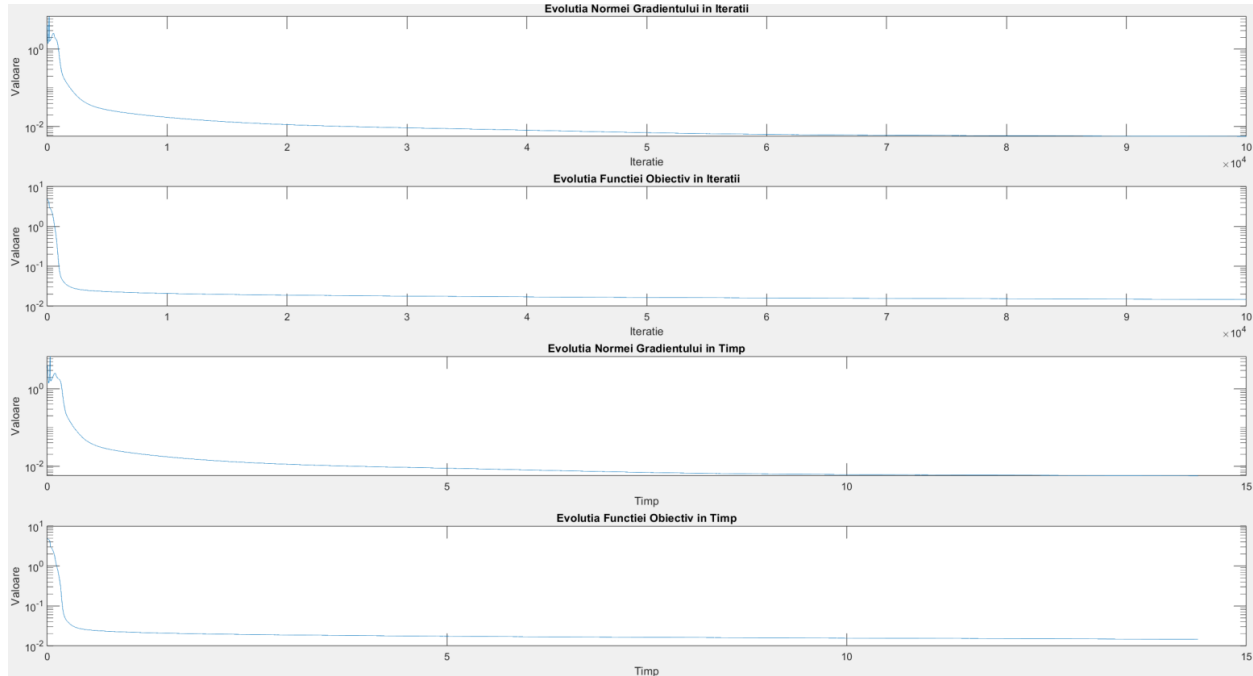
$$L(e, y) = \frac{1}{2N} \|e - y\|^2 = \frac{1}{2N} \sum_{i=1}^N (e_i - y_i)^2$$

Așadar, problema de minimizare fără constrângeri se transformă în:

$$\min_{x \in \mathbb{R}^m, X \in \mathbb{R}^{(n+1) \times m}} \frac{1}{2N} \|g(\bar{A}X) x - e\|^2$$

Rezolvarea problemei de minimizare prin Metoda Gradient:

Am implementat în MatLab metoda Gradient cu pasul de învățare constant $\alpha_k = 10^{-3}$ și am inițializat ponderile x și X cu randn() * 0.1, pentru a le limita magnitudinile.



Performanța rețelei se verifică și prin indicatorii $R^2 = 0.9616$ și $MSE = 0.051$.

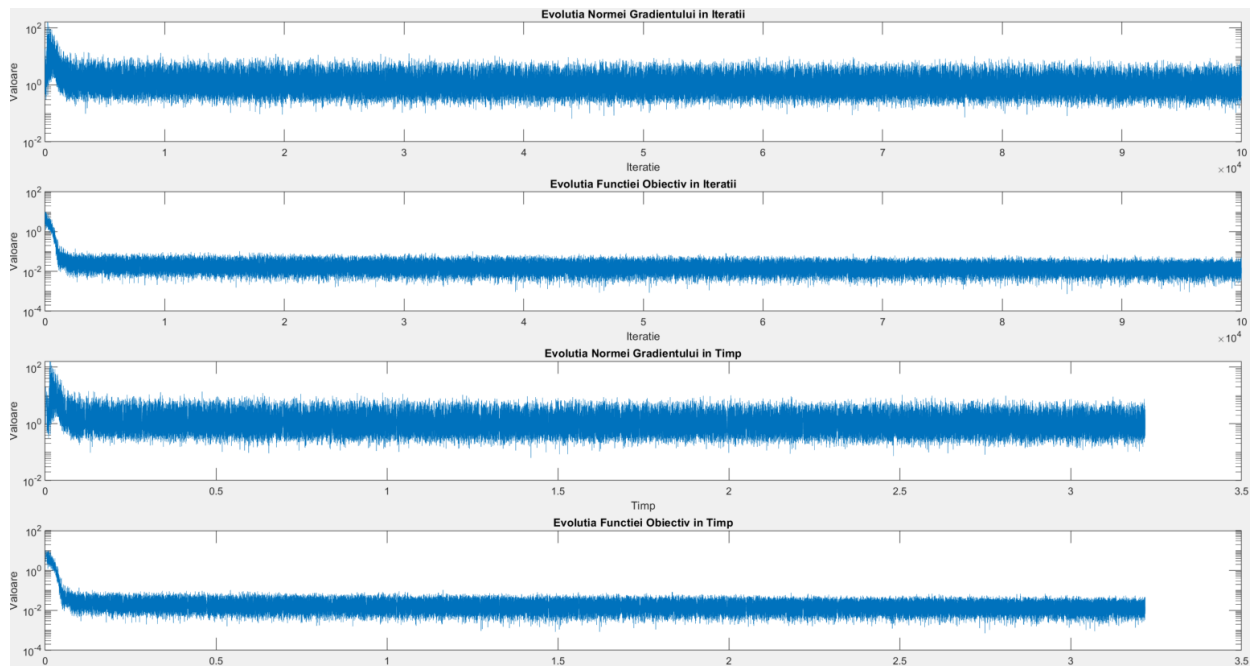
Exemple de etichete vs. rezultate din baza de date de test:

3.7000	3.4628
1.8000	1.6197
1.6000	1.7026
1.9000	1.9179
2.0000	1.9624
2.2000	2.1277
2.1000	2.0565
2.3000	2.2892
2.6000	2.3417
3.8000	3.5543
4.4000	4.5148
4.5000	4.4443
3.3000	2.8785
2.0000	1.8701
1.1000	1.1234
0.9000	1.0141
0.8000	0.8430
0.7000	0.7254
0.5000	0.1767
0.3000	0.2353
0.5000	0.4660
1.1000	1.0978
3.0000	3.4217
3.8000	4.8306
3.4000	3.5404

În concluzie, curbele de descreștere ale normei gradientului și funcției obiectiv sunt conform așteptărilor metodei Gradient, iar valorile prezise sunt suficient de apropiate de etichete, judecând valorile lor numerice și valorile indicilor R^2 și MSE.

Rezolvarea problemei de minimizare prin Metoda Gradient Stochastic:

Am implementat metoda Gradient Stochastic cu aceiași parametri ca în metoda Gradient, diferența fiind că am selectat random, la fiecare pas, un anumit număr de exemple pentru antrenarea rețelei neurale. Empiric am constatat că pentru aprox. 10 exemple, rețeaua neurală furniza cele mai bune rezultate.



și $R^2 = 0.9638$, $MSE = 0.0481$.

Exemple de etichete vs. rezultate din baza de date de test:

3.7000	3.4417
1.8000	1.6422
1.6000	1.7239
1.9000	1.9376
2.0000	1.9823
2.2000	2.1383
2.1000	2.0688
2.3000	2.2896
2.6000	2.3467
3.8000	3.5013
4.4000	4.4604
4.5000	4.3844
3.3000	2.8721
2.0000	1.8954
1.1000	1.1514
0.9000	1.0423
0.8000	0.8720
0.7000	0.7532
0.5000	0.2506
0.3000	0.2720
0.5000	0.5070
1.1000	1.1476
3.0000	3.4084
3.8000	4.7995
3.4000	3.5305

În concluzie, observăm că indicii R^2 și MSE sunt puțin mai buni față de metoda Gradient, rezultat neintuitiv, dar care se verifică cu noțiunile predate în curs. Evoluțiile gradientului și funcției obiectiv sunt mai haotice, ca urmare a alegerii random la fiecare pas a unui batch de exemple de test. În plus, rezultatele mai bune se obțin și într-un timp de aproape 5 ori mai scurt (a se observa graficele în funcție de timp, metoda Gradient execută pașii în aproape 15 secunde, în timp ce metoda Gradient Stochastic execută același număr de pași în puțin peste 3 secunde).

Anexa cu programul MatLab:

(se execută mai întâi secțiunea de citire a datelor, după care, la alegere, secțiunea metodei Gradient sau cea a metodei Gradient Stochastic)

```
%% CITIREA DATELOR SI INITIALIZARI
```

```
clear;
```

```
clc;
```

```
% Extrag datele din fisierul .csv
```

```
opts = detectImportOptions('AirQualityUCI.csv', 'Delimiter',';', 'DecimalSeparator',',',  
'VariableNamesLine',1, 'VariableNamingRule','preserve');
```

```
opts.DataLines = [2, 350];
```

```
opts.SelectedVariableNames = opts.VariableNames(3:end);
```

```
setDate = readtable('AirQualityUCI.csv', opts);
```

```
setDate = table2array(setDate);
```

```
% Elimin seturile care au lipsuri in vectorul de etichete
```

```
valid = setDate(:,1) ~= -200;
```

```
setDate = setDate(valid, :);
```

```
% Izolez vectorul de etichete
```

```
e = setDate(:,1);
```

```
setDate = setDate(:, 2:end);
```

```
[nrTotalExemple, nrCaract] = size(setDate);
```

```
% Impart baza de date in baza de antrenare si de testare
```

```
procentAntrenare = 0.8;
```

```
procentTest = 1 - procentAntrenare;
```

```
nrExempleAntrenament = floor(nrTotalExemple * procentAntrenare);
```

```
nrExempleTest = nrTotalExemple - nrExempleAntrenament;
```

```
A = setDate(1 : nrExempleAntrenament, :);
```

```
exempleTest = setDate(nrExempleAntrenament + 1 : end, :);
```

```
eAntrenament = e(1:nrExempleAntrenament);
```

```
eTest = e(nrExempleAntrenament + 1 : end);
```

```
% Inlocuiesc lipsurile cu valoarea medie a caracteristicii
```

```
A(A == -200) = NaN;
```

```
medie = mean(A, 1, 'omitnan');
```

```
for j = 1:nrCaract
```

```
    poz = isnan(A(:,j));
```

```
    A(poz,j) = medie(j);
```

```
end
```

```
exempleTest(exempleTest == -200) = NaN;
```

```
for j = 1:nrCaract
```

```
    poz = isnan(exempleTest(:,j));
```

```
    exempleTest(poz,j) = medie(j);
```

```
end
```

```
% Standardizare
```

```
mu = mean(A, 1);
```

```
sigma = std( A, [], 1);
```

```
A = (A - mu) ./ sigma;
```

```
exempleTest = (exempleTest - mu) ./ sigma;
```

```
% Adaug coloana de 1 pentru bias
```

```
AExtins = [A, ones(nrExempleAntrenament,1)];
```

```
exempleTest = [exempleTest, ones(nrExempleTest,1)];
```

```
% Initializari
```

```
a = 1;
```

```
g = @(z) a * (z.^3);
```

```
g_prim = @(z) 3 * a * (z.^2);
```

```
g_secund = @(z) 6 * a * z;
```

```
nrNeuroni = 20;
```

```
rng(0);
```

```
x = randn(nrNeuroni, 1) * 0.1;
```

```
X = randn(nrCaract + 1, nrNeuroni) * 0.1;
```

```
%% METODA GRADIENT
```

```
iter = 1;
```

```
maxIter = 100000;
```

```
evolutieNorma = zeros(maxIter, 1);
```

```
evolutieFuncOb = zeros(maxIter, 1);
```

```
timp = zeros(maxIter, 1);
```

```
err = 1e-6;
```

```
pas = 1e-3;
```

```
start = tic;
```

```
while iter <= maxIter
```

```
    U = AExtins * X;
```

```
    Z = g(U);
```

```
    y = Z * x;
```

```
    r = y - eAntrenament;
```

```
    gradient_X = (1/nrExempleAntrenament) * AExtins' * ( (r * x') .* g_prim(U) );
```

```
    gradient_x = (1/nrExempleAntrenament) * Z' * r;
```

```
    timp(iter) = toc(start);
```



```
evolutieFuncOb(iter) = (1 / (2*nrExempleAntrenament)) * norm(r)^2;
```

```
norma = norm([gradient_X(:); gradient_x]);
```

```
evolutieNorma(iter) = norma;
```

```
if norma < err
```

```
    break;
```

```
end
```

```
X = X - pas * gradient_X;
```

```
x = x - pas * gradient_x;
```

```
iter = iter + 1;
```

```
end
```

```
figure('Name', 'METODA GRADIENT');
```

```
subplot(4, 1, 1);
```

```
semilogy(evolutieNorma);
```

```
title('Evolutia Normei Gradientului in Iteratii');
```

```
xlabel('Iteratie');
```

```
ylabel('Valoare');
```

```
subplot(4, 1, 2);
```

```
semilogy(evolutieFuncOb);
```

```
title('Evolutia Functiei Obiectiv in Iteratii');
```

```
xlabel('Iteratie');  
ylabel('Valoare');
```

```
subplot(4, 1, 3);  
semilogy(timp, evolutieNorma);  
title('Evolutia Normei Gradientului in Timp');  
xlabel('Timp');  
ylabel('Valoare');
```

```
subplot(4, 1, 4);  
semilogy(timp, evolutieFuncOb);  
title('Evolutia Functiei Obiectiv in Timp');  
xlabel('Timp');  
ylabel('Valoare');
```

```
yTest = g(exempleTest * X) * x;  
rTest = yTest - eTest;  
medie_eTest = mean(eTest);
```

```
disp('Indicatori de performanta pentru Metoda Gradient:');  
R2 = 1 - sum( rTest.^2 ) / sum( (eTest - medie_eTest).^2 )  
MSE = (1/nrExempleTest) * sum( rTest.^2 )
```

```
%% METODA GRADIENT STOCHASTIC
```

```
iter = 1;
```

```

maxIter = 100000;

evolutieNorma = zeros(maxIter, 1);
evolutieFuncOb = zeros(maxIter, 1);
timp = zeros(maxIter, 1);

err = 1e-6;
pas = 1e-3;

dimensiuneBatch = 10;

start = tic;
while iter <= maxIter

    poz = randperm(nrExempleAntrenament, dimensiuneBatch);
    ABatch = AExtins(poz, :);
    eBatch = eAntrenament(poz);

    U = ABatch * X;
    Z = g(U);
    y = Z * x;
    r = y - eBatch;

    gradient_X = (1/dimensiuneBatch) * ABatch' * ( (r * x') .* g_prim(U) );
    gradient_x = (1/dimensiuneBatch) * Z' * r;

    timp(iter) = toc(start);

```

```
evolutieFuncOb(iter) = (1 / (2*dimensiuneBatch)) * norm(r)^2;
```

```
norma = norm([gradient_X(:); gradient_x]);
```

```
evolutieNorma(iter) = norma;
```

```
if norma < err
```

```
    break;
```

```
end
```

```
X = X - pas * gradient_X;
```

```
x = x - pas * gradient_x;
```

```
iter = iter + 1;
```

```
end
```

```
figure('Name', 'METODA GRADIENT STOCHASTIC');
```

```
subplot(4, 1, 1);
```

```
semilogy(evolutieNorma);
```

```
title('Evolutia Normei Gradientului in Iteratii');
```

```
xlabel('Iteratie');
```

```
ylabel('Valoare');
```

```
subplot(4, 1, 2);
```

```
semilogy(evolutieFuncOb);
```

```
title('Evolutia Functiei Obiectiv in Iteratii');
```

```
xlabel('Iteratie');  
ylabel('Valoare');
```

```
subplot(4, 1, 3);  
semilogy(timp, evolutieNorma);  
title('Evolutia Normei Gradientului in Timp');  
xlabel('Timp');  
ylabel('Valoare');
```

```
subplot(4, 1, 4);  
semilogy(timp, evolutieFuncOb);  
title('Evolutia Functiei Obiectiv in Timp');  
xlabel('Timp');  
ylabel('Valoare');
```

```
yTest = g(exempleTest * X) * x;  
rTest = yTest - eTest;  
medie_eTest = mean(eTest);
```

```
disp('Indicatori de performanta pentru Metoda Gradient Stochastic:');
```

```
R2 = 1 - sum( rTest.^2 ) / sum( (eTest - medie_eTest).^2 )
```

```
MSE = (1/nrExempleTest) * sum( rTest.^2 )
```